

MergeTree

Basándose en el concepto de los árboles [LSM](#), los datos de una [tabla de la familia MergeTree](#) se almacenan en fragmentos horizontales denominados *parts* que, posteriormente, se fusionan en segundo plano mediante un subproceso dedicado. Cada fragmento tiene normalmente su propio directorio y los datos se organizan **de forma columnar ordenados según la clave primaria**



ClickHouse vs Parquet

El enfoque columnar difiere del formato columnar de *Parquet*, *DuckDB*, *Snowflake* o *BigQuery*, donde los datos se particionan primero horizontalmente en subconjuntos de filas y, dentro de cada subconjunto, las columnas se almacenan muy juntas. En ClickHouse, **la tabla solo se fragmenta verticalmente** y cada columna se almacena por separado

En los casos en que se escriben partes pequeñas, almacenar los datos en varios archivos separados para cada columna puede afectar negativamente al rendimiento de lectura y escritura. Por lo tanto, *MergeTree* ofrece dos formatos para escribir columnas:

- Amplio: Los datos se escribirán en archivos separados, cada uno correspondiente a una columna
- Compacto: Con una parte pequeña (por defecto, menor de 10 MB), las columnas se almacenan consecutivamente en un único archivo para aumentar la localidad espacial en las lecturas y escritura

Ejemplo básico

El código siguiente muestra el *DDL* de creación de una tabla *MergeTree* en *ClickHouse*

```
CREATE TABLE ejMergeTree (  
  id UInt16,  
  created_time Date,  
  comment String  
) ENGINE = MergeTree()  
  ORDER BY (id, created_time)  
  PARTITION BY toYYYYMM(created_time)  
  TTL created_time + INTERVAL 3 MONTH;
```



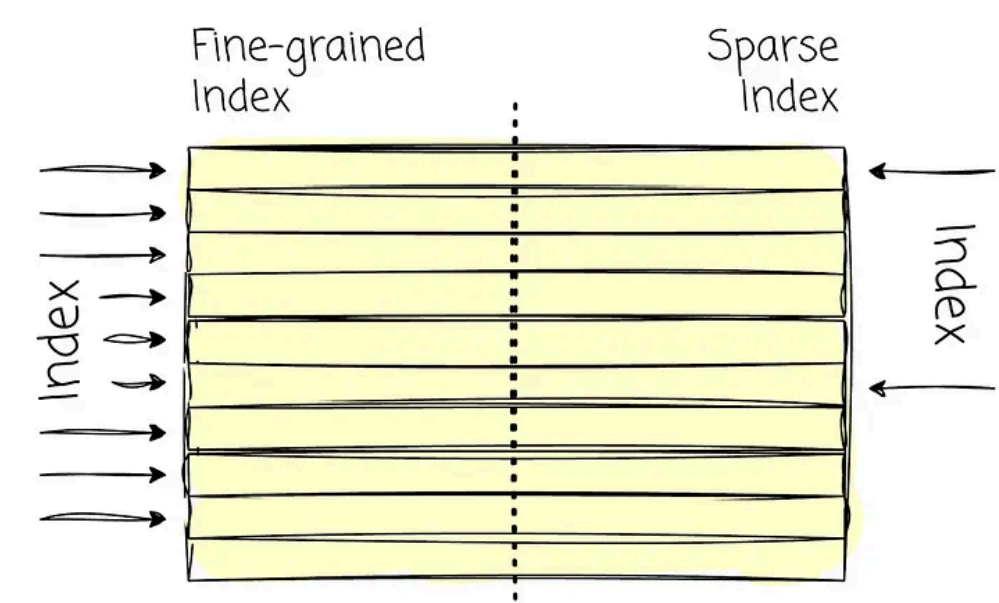
Es importante notar los siguientes **aspectos principales**:

1. **No hay clave primaria** explícita (asume las columnas en *order by*)
2. Consecuentemente, las claves primarias **no garantizan la unicidad**. Es decir, puede haber múltiples filas con idéntica clave primaria
3. Todas las **escrituras son secuenciales e inmediatas**
4. Almacena de manera **ordenada** ya que el reparticionado de los datos ocurre *en diferido*
5. El valor del *TTL* permite establecer un tiempo de **caducidad de los datos**



Indexación

A diferencia de una base de datos *OLTP*, los registros **no tienen un índice asociado a la clave primaria**. En su lugar, el *índice de ClickHouse* apunta a un rango de datos (*Sparse Index*, índice disperso). De este modo, *ClickHouse* puede aprovechar los índices para mejorar el rendimiento de lectura. Dado que los datos están ordenados, el índice disperso permite reducir el espacio de búsqueda durante la búsqueda binaria. Este enfoque también evita que el proceso de indexación afecte al rendimiento de escritura pues sería muy costoso si se ingirieran millones de registros manteniendo un índice para cada uno de ellos



La *cláusula PARTITION BY* permite agrupar los datos de las tablas de la familia de motores *MergeTree* en unidades lógicas organizadas, lo que constituye una forma de organizar los datos que resulta conceptualmente significativa y se ajusta a criterios específicos, como intervalos de tiempo, categorías u otros atributos clave. Estas unidades lógicas facilitan la gestión, la consulta y la optimización de los datos



Esquema de particionado

ClickHouse solo fusiona partes de datos **dentro de una misma partición pero no entre particiones**. Esto significa que las partes que pertenecen a particiones diferentes nunca se fusionan. Por consiguiente, si se elige una clave de particionado con una cardinalidad elevada las partes repartidas entre miles de particiones nunca podrán fusionarse. En caso extremo, el tamaño podría superar los límites preconfigurados y aparecería el error *Too many parts*. Normalmente ello de nota un esquema de particionado incorrecto pero, en todo caso, la solución es muy sencilla: elige una clave de partición razonable (cardinalidad entre 1000 y 10000)

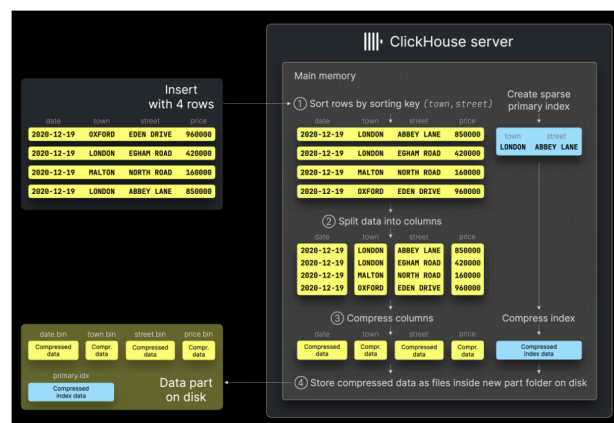
Modelo de Almacenamiento

Funcionamiento *MergeTree*

1. Primero ordena, almacena después
2. Las filas se almacenan en bloques (gránulos/parts) **inmutables**
3. Los bloques, además de los datos, contienen los metadatos de manera que no se requiere un catálogo de metadatos
4. El número de filas n cada bloque es 8192. Este valor es el denominado **index granularity**



Ingestión de Datos

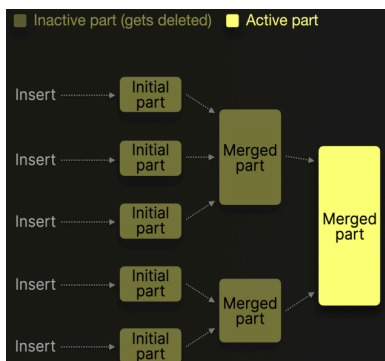


Modelo de Ejecución en Background

Para gestionar el número de *parts* de cada tabla *ClickHouse* ejecuta periódicamente en segundo plano dos **procesos MS y ME** combinando las partes más pequeñas en otras más grandes hasta que alcanzan un tamaño comprimido configurable (normalmente, unos 150 GB). Las partes fusionadas se marcan como inactivas y se eliminan tras un intervalo de tiempo configurable. Con el paso del tiempo, este proceso crea una **estructura jerárquica de partes fusionadas** que se denomina **tabla MergeTree**

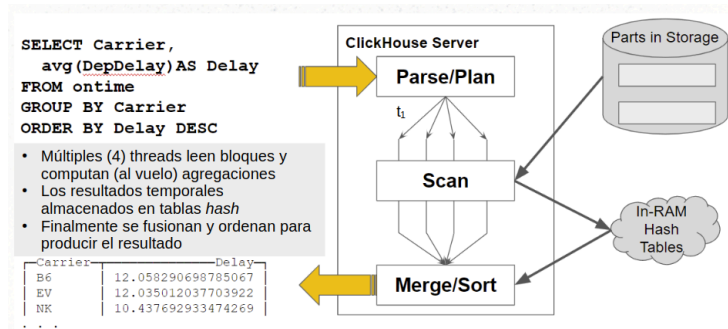
MergeSelector (MS) y MergeExecutor (ME)

- MS revisa tablas periódicamente y planifica fusiones
- ME selecciona y ejecuta fusiones



Agregados *on-the-fly*

- Agregados acumulados en memoria (tablas hash)
- Resultado: Merge/Sort



Scan/Merge

El procesamiento de los agregados se ejecuta en paralelo sobre cada *part*. En el ejemplo del cálculo del retardo medio que muestra la consulta anterior (figura a la derecha), cuatro *threads* leen simultáneamente los bloques de datos y realizan una agregación inicial al vuelo, sobre la marcha

Durante la fase de escaneo se contabiliza el número de datos (denominador) y se calcula en cada trozo la suma acumulada de los retardos (numerador). Los resultados parciales se acumulan en tablas *hash* donde hay una clave por cada valor de *GROUP BY*. Una vez finalizado el escaneo, es necesario fusionar todos los agregados y ordenarlos. La imagen a continuación muestra gráficamente cómo funciona

